Excel | SQL | Python | Tableau

Telecom Network & Customer Experience Analytics

End-to-End Data Analyst Project Report

PREPARED BY	Mohan Srinivas
DEGREE	B.E. Computer Science & Engineering
DOMAIN	Telecom | Customer Analytics | Network Performance
TOOLS	Excel | SQL Server | Python (Pandas) | Tableau
YEAR	2024

CONTENTS

Table of Contents

01 / OVERVIEW

Project Overview & Business Context

This project is an end-to-end data analytics case study built around a telecom company dealing with three growing problems: rising customer complaints about network quality, an increase in delayed and unpaid bills, and uncertainty about whether their 5G rollout was actually making a difference for customers. The goal was to act as a data analyst — take raw data across four business areas and turn it into clear, actionable insights that leadership could use to make investment and retention decisions.

The project was deliberately designed to cover the full analyst workflow: cleaning and preparing data in Excel, structuring it in a relational database and querying it in SQL, building a customer risk scoring model in Python, and presenting everything through Tableau dashboards built for different business audiences.

Business Problems Addressed

Network Quality & Complaints

Customers were raising issues but there was no clear picture of which cities were worst affected, what types of issues were most common, or how long resolution was taking on average.

Payment Delays & Churn Risk

A significant number of bills were being paid late or not at all. Leadership suspected this was connected to poor service experience — a major churn signal that needed to be confirmed with data.

5G Investment Validation

The company had begun rolling out 5G and needed evidence to confirm whether 5G customers were experiencing meaningfully fewer issues compared to 4G and 3G users.

500+	4	4	3
Customer Records	Tables Analyzed	Tools Used	Business Questions

02 / DATASET

Dataset Description

The dataset covers four related operational tables, all linked through a shared **customer_id**. This structure mirrors how data is stored in real telecom companies — in separate systems that need to be joined for full analysis.

Table	Records	Key Columns	Purpose
customers	500+	customer_id, customer_type, age, city, signup_date	Demographics and subscription type (Prepaid / Postpaid)
network_usage	Multiple per customer	data_used_gb, call_minutes, network_type	Daily usage — data consumed and call activity
network_issues	Variable	issue_type, resolution_time_hrs, issue_date, resolved	Complaints logged and resolution tracking
billing	Monthly	bill_amount, payment_status, bill_month	Billing history and payment behaviour per customer

Coverage Details

Cities	Bangalore, Mumbai, Delhi, Chennai, Kolkata, Hyderabad
Customer Types	Prepaid and Postpaid
Network Types	3G, 4G, 5G
Time Period	Monthly billing and daily usage data across 2023-2024
Relationships	All tables linked via customer_id (Primary / Foreign Key)

03 / PHASE 1

Excel — Data Cleaning & Exploration

The first step in any end-to-end project is making sure the raw data is reliable. Before any SQL queries or Python code was written, the dataset was cleaned and validated in Excel. This phase is often underestimated but it is the most important — dirty data leads to wrong conclusions regardless of how well the later analysis is done.

Tasks Completed

#	Task	Outcome
1	Cleaned and standardized city names	Removed inconsistent capitalization and whitespace — e.g. 'bangalore' vs 'Bangalore'
2	Standardized date formats	Ensured all date columns used a consistent format for downstream SQL and Python use
3	Created Month and Year columns	Extracted month and year from date fields using Excel formulas for time-series grouping
4	High-Usage Customer flag (IF logic)	Used IF formula to label customers exceeding data or call thresholds as 'High Usage: Yes'
5	Pivot Tables	Built pivot tables for city-wise issue counts, customer type distribution, and monthly billing totals

Why this phase matters

Standardizing city names alone prevents SQL GROUP BY from treating 'Mumbai' and 'mumbai' as two separate cities — which would completely distort any city-level analysis. Excel was also used to sense-check the data: confirming row counts, spotting obvious outliers, and validating that categories were consistent before importing into SQL.

04 / PHASE 2

SQL — Database Design & Analysis

With clean data ready, the next phase was to build a relational database in SQL Server and write structured analysis queries. SQL is the core tool for most data analyst roles, so this phase covered both foundational skills (schema design, joins, aggregations) and advanced techniques (window functions).

Database Schema

Four tables were created with **PRIMARY KEY** and **FOREIGN KEY** constraints to enforce data integrity. All data was loaded into the database using **BULK INSERT** from CSV files.

SQL Queries Written

#	Task	Outcome
1	Multi-table JOIN across all 4 tables	Unified view combining customer demographics, usage, complaints, and billing
2	Complaint Rate per Customer	Total Complaints / Total Usage Records — identifies highest-risk customers
3	Monthly Complaint Trend	Aggregated complaint count by month and year to detect seasonal or worsening trends
4	Running Total Complaints	SUM() OVER (ORDER BY month) window function for a cumulative complaint trend line
5	Month-over-Month City Complaints	LAG() window function to show change vs previous month, partitioned by city
6	City-wise Issues and Avg Resolution Time	Directly answered: which cities need the most urgent network investment?
7	Payment Delays vs Complaint Count	Cross-tab of complaints and delayed payments per customer to test the churn hypothesis
8	5G vs 4G/3G Issue Comparison	Grouped total issues by network_type to measure 5G performance vs older networks

View Created

A reusable SQL View was created to summarize total complaints and average resolution time per customer. This was used as a data source in Tableau.

```
CREATE VIEW VW_customer_complaint_analysis — total_complaints, avg_resolution_time_hrs
grouped by customer_id and city
```

Window Functions Used

SUM() OVER() for running totals, LAG() for month-over-month difference, and PARTITION BY to separate city-level trends. These are among the most commonly tested SQL concepts in data analyst interviews and demonstrate beyond-basic SQL proficiency.

05 / PHASE 3

Python — EDA & Customer Risk Scoring

Python (Pandas) served three purposes in this project: thorough data validation to confirm the cleaned Excel data was correct, building a custom customer risk scoring model, and generating exploratory visualizations. This phase demonstrates practical Python skills including lambda functions, custom if-else scoring logic, and multi-dataframe merging.

Data Validation

#	Task	Outcome
1	Shape and column checks	Confirmed row counts and column names across all 4 tables
2	Null value detection	isnull().sum() on all tables — confirmed no missing values in any column
3	Duplicate row detection	duplicated().sum() check across all tables
4	Data type conversion	Converted all date columns to datetime using pd.to_datetime()
5	Negative value check	Verified no negative data_used_gb or bill_amount values exist
6	Category validation	Printed unique values for city, customer_type, issue_type, and payment_status

Feature Engineering

1. Heavy Usage Flag (Lambda Function)

Customers using more than 10 GB of data OR more than 200 call minutes in a single record were flagged as heavy users using an apply + lambda approach:

```
heavy_usage_flag = 1 if (data_used_gb > 10 or call_minutes > 200) else 0
```

2. Customer Risk Score (Custom Function)

A scoring function combined four factors into a single risk number per customer. Each factor contributed points based on thresholds established from the data:

Factor	Condition	Points
Total Complaints	> 5 complaints	+2
Avg Resolution Time	> 48 hours	+2

Payment Risk	Any delayed / unpaid bill	+2
Heavy Usage	Flagged as heavy user	+1

High Risk	Medium Risk	Low Risk
Score >= 5	Score 3 - 4	Score < 3

EDA Visualizations Created

- Customer distribution by city (bar chart)
- Customer type split — Prepaid vs Postpaid (pie chart)
- Total data usage by network type (bar chart)
- Average call minutes by network type (bar chart)
- Most common network issue types (bar chart)
- Average resolution time by issue type (bar chart)
- Payment status distribution (bar chart)

The risk score output was exported as **customer_risk_analysis.csv** and used as a data source in the Tableau dashboard for the churn indicators view.

06 / PHASE 4

Tableau — Dashboard Design

The final phase was building dashboards that translated the entire analysis into a format that non-technical stakeholders can understand and act on. Three dashboards were built, each designed for a different audience and business question.

Dashboard 1: Executive KPI Dashboard

Top-level summary for senior leadership. Shows the most critical numbers at a glance: total complaints, total revenue, percentage of delayed payments, and count of high-risk customers. Designed for a 30-second read with no scrolling needed.

Dashboard 2: City-wise Network Performance

Breaks down network issues, average resolution times, and complaint volumes by city. This dashboard directly answers which cities need urgent investment and shows how different cities compare on both complaint volume and resolution speed.

Dashboard 3: Customer Risk & Churn Indicators

Uses the Python risk score output to visualize the distribution of High, Medium, and Low Risk customers. Shows the relationship between complaint frequency and payment delays — the two strongest churn predictors found in the analysis.

Dashboard design principle

Each dashboard was built around one business question and one target audience. A dashboard that tries to show everything ends up communicating nothing. The goal was: executives get KPIs, network teams get city performance, retention teams get risk and churn signals.

07 / INSIGHTS

Business Questions & Insights

These three questions were the reason this project existed. Everything from the Excel cleaning to the SQL queries to the Python risk model was built to answer them. Below are the findings from each.

Q Which cities require urgent network improvement?

A The SQL analysis grouped total complaint counts and average resolution times by city. Cities appearing at both the top of the complaint count ranking AND with the longest average resolution times were flagged as highest priority for infrastructure investment. The Tableau city-wise dashboard made these patterns visible at a glance for the operations team.

Q Does poor network quality affect payments and churn?

A Yes — and the data showed a clear pattern. Customers with more than 5 complaints also showed a significantly higher rate of delayed or unpaid bills. This was confirmed in SQL (complaint count vs delayed payment count per customer) and reinforced by the Python risk model, where payment_risk and total_complaints were the two highest-weighted churn factors. Poor network experience directly erodes customer trust and payment behaviour.

Q Are 5G users experiencing fewer issues than 4G/3G users?

A The network_type grouping in SQL compared total issue counts across 3G, 4G, and 5G. This allowed a direct, data-backed comparison to either validate or challenge the assumption that 5G delivers a better customer experience — giving the network team evidence to support or reassess their rollout investment strategy.

Q What additional patterns did the EDA reveal?

A A few extra findings emerged during the Python EDA: heavy usage customers had a higher likelihood of appearing in the High Risk category even when they had fewer complaints, suggesting that network strain from heavy data consumers may be an early churn signal worth monitoring separately. Postpaid customers showed higher average bill amounts but also slightly higher complaint rates, pointing to a potential service expectation gap for the company's premium subscriber segment.

Q How reliable is the risk scoring model?

A The risk model was built using business logic rather than machine learning — which makes it transparent and explainable to stakeholders. Each point in the score comes from a clear, defensible threshold (e.g. more than 5 complaints, resolution longer than 48 hours). For a first analysis, this approach is more trustworthy and easier to communicate than a black-box model.

08 / LEARNINGS

Key Takeaways & Learnings

This project was as much about developing analyst thinking as it was about technical execution. The five most important lessons from building this end-to-end:

1. Each tool has a specific job

Excel for quick cleaning and sense-checking. SQL for structured querying at scale. Python for logic-heavy calculations and scoring. Tableau for communication. Choosing the right tool for the right task is itself a skill — and switching between them fluently is what end-to-end analysis actually looks like.

2. Data cleaning takes longer than expected — and matters more

The Excel phase felt like a small step but was foundational. Even minor inconsistencies in city names caused major issues in SQL GROUP BY results. Real-world data is always messier than it appears, and rushing through cleaning always costs more time later.

3. Window functions change what questions you can ask

Basic aggregations answer 'what happened'. Adding LAG() and running totals with SUM() OVER() made it possible to ask 'how is this changing over time' — which is what business leaders actually want to know.

4. Risk scoring bridges analysis and action

Grouping customers into High / Medium / Low Risk made the Python output immediately usable for a business audience. Raw numbers are hard to act on — categories make prioritization easy. This is what 'so what' looks like in practice.

5. Dashboards are for decisions, not for data

The biggest Tableau lesson was restraint. A good dashboard doesn't show everything — it answers one question per view. The executive dashboard was deliberately stripped down to the numbers that matter most for a 30-second leadership read.

09 / SKILLS

Tools & Skills Summary

Category	Skill / Concept	Used In
Data Cleaning	Standardization, date formatting, IF logic	Excel
Database Design	Schema creation, PKs, FKs, BULK INSERT	SQL
SQL Querying	Multi-table JOINs, GROUP BY, aggregations	SQL
SQL Advanced	Window functions: LAG(), SUM() OVER(), PARTITION BY	SQL
Data Validation	Null checks, duplicates, dtype checks, constraints	Python
Feature Engineering	Lambda functions, custom scoring functions	Python
EDA	Distribution analysis, groupby aggregations	Python
Visualization	Bar charts, pie charts, Matplotlib, Seaborn	Python
Dashboard Design	KPI views, filters, calculated fields	Tableau
Business Thinking	Translating findings into recommendations	All Phases

A note on this project

This was built as a portfolio project while learning data analytics as a fresher. Every phase was treated as a real-world task — not just completing exercises, but thinking about why each step matters and what the business would actually need. The goal was to demonstrate not just technical skills, but the mindset of a working data analyst.